

TRACERTL: Agile Performance Evaluation for Microarchitecture Exploration

Zifei Zhang^{†‡}, Yanan Xu[†], Sa Wang^{†‡}, Dan Tang^{†§}, Yungang Bao^{†‡}

State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, China[†];

University of Chinese Academy of Sciences, China[‡];

Beijing Institute of Open Source Chip, China[§]

{zhangzifei20z,xuyinan,wangsa,tangdan,baoyg}@ict.ac.cn

Abstract—While agile chip development methodologies have accelerated RTL design and simulation, performance evaluation remains constrained by three challenges: (1) inefficient feature prototyping caused by the tight coupling between functional correctness and performance evaluation, particularly for large-scale, error-prone microarchitectures; (2) limited workloads due to incomplete peripheral/software environments or unavailable source code; and (3) time-consuming warm-up phases in sampling-based simulation, required to mitigate cold-start effects.

To address these challenges, we propose TRACERTL, an agile, trace-driven performance evaluation methodology that decouples the functional and performance components of CPU RTL designs. It introduces three techniques: (1) a trace-driven performance exploration framework that bypasses full functional correctness while preserving performance accuracy; (2) a trace transformation technique, TraceBridge, that replays traces across different formats and instruction sets; and (3) a fast warm-up strategy, TraceDedup, that eliminates redundant traces and efficiently initializes microarchitectural states.

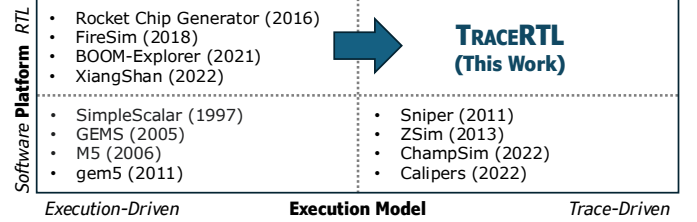
Using TRACERTL, we develop the first trace-driven RTL CPU derived from XiangShan, a high-performance out-of-order RISC-V processor. TRACERTL achieves performance accuracies of 99.87% and 99.86% on SPECint2017 and SPECfp2017, respectively. With TraceBridge, we evaluate x86-based Google workload traces on a RISC-V RTL CPU and reveal distinct memory-bound behavior. TraceDedup further accelerates warm-up phases in sampling-based simulations by $1.5\times$ to $11.8\times$.

I. INTRODUCTION

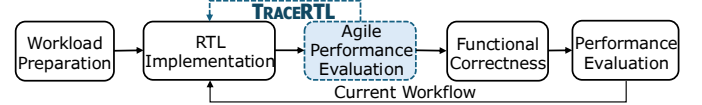
Performance has always been a central consideration in CPU development. As Moore’s Law slows and application demands diversify, achieving further performance improvements has become increasingly challenging. This highlights the importance of microarchitecture exploration methodologies. A key question is: given a baseline CPU design, how can we efficiently quantify the performance impact of a proposed hardware feature under representative workloads?

Among available performance evaluation methods, the most faithful is to use the register-transfer level (RTL) implementation, written in a hardware description language. As the definitive description of the microarchitecture, RTL is the most reliable basis for assessing CPU design changes. Ultimately, any proposed feature must be implemented and evaluated in RTL to determine its true performance impact.

However, since RTL simulation is slow and its development process is time-consuming, the computer architecture community has adopted more efficient approaches to accelerate



(a) Approaches to microarchitecture exploration.



(b) Current and TRACERTL performance evaluation workflows.

Fig. 1: Microarchitecture exploration methods and workflows.

early-stage exploration before realizing a proposed feature in RTL. As shown in Fig. 1a, software-based architectural simulators [11], [12], [16], [17], [30], [33], [44], [57] model low-level hardware components through high-level languages and abstractions, enabling fast simulations and rapid design iteration. Despite their high productivity in *early-stage* exploration, the *last mile* remains unavoidable: performance must still be re-evaluated on RTL after initial simulator studies, since the additional modeling layer inevitably introduces discrepancies that require substantial engineering efforts and costly calibration to the actual implementation [49].

Given the high last-mile cost of architectural simulators, recent advancements in RTL design and simulation offer strong potential for a seamless, progressive refinement workflow from early-stage exploration to the last mile. On the design side, high-level hardware construction languages [7], [13], [74] enable parameterized and reusable components, allowing rapid implementation and iteration of new microarchitectural ideas. On the evaluation side, efficient RTL simulation methods [26], [66], [68], [69], [76], especially FPGAs and emulators [25], [27], [37], [38], significantly accelerating large-scale RTL simulation. These capabilities have already been demonstrated in several open-source, industrial-competitive CPUs [19]–[21], [70], [75], which provide realistic and accessible microarchitecture research platforms [8], [29], [34], [67], [71]. For example, it takes less than 200 minutes and approximately

300 lines of modified Chisel code to implement an instruction scheduler policy PUBS [4] on XiangShan, a high-performance RISC-V CPU achieving >15 SPECint2006/GHz [70], [73].

These trends motivate us to adapt proven exploration techniques from simulators directly to RTL, aiming to *inherit the agile workflow of simulator-based exploration while enabling a seamless integration to RTL for last-mile evaluation*.

To realize this opportunity, we *propose* **TRACERTL**, an RTL-based performance evaluation methodology that derives a trace-driven RTL model from an existing execution-driven RTL implementation. As illustrated in Fig. 1a, TRACERTL reuses open-source, silicon-proven RTL designs as a solid foundation for faithful microarchitecture exploration. It drives performance-critical modules with pre-generated traces, enabling simulator-like agility for early-stage exploration on RTL. By deriving the trace-driven model directly from RTL, it inherently avoids costly last-mile calibration and preserves performance fidelity for evaluation of the proposed feature.

One key motivation behind TRACERTL is to overcome the execution-driven nature of current RTL designs. As illustrated by the white boxes in Fig. 1b, the modified CPU must first pass full functional verification *before* any performance evaluation can be conducted. This tight coupling forces every RTL design change to undergo complete implementation, verification, and lengthy simulations, even when unrelated to performance. For example, evaluating optimizations for virtualized, two-stage address translation [54] requires implementing complex privileged operations to guarantee correct functionality, whose functional details, however, do not affect the performance.

With TRACERTL, this strict dependency between functional correctness and performance evaluation is eliminated. As highlighted in Fig. 1b, TRACERTL enables *agile performance evaluation* without first implementing or verifying unrelated functional details. Additionally, it accepts trace inputs from *a broader range of real-world applications*, including those with unavailable source code, different ISAs, or peripheral dependencies [1]–[3], [64], without requiring porting into an RTL simulation. Furthermore, it *reduces simulation time* by removing redundant warm-up instructions through offline trace analysis, which is particularly important for sampling-based simulation [18], [56], [62]. To realize these capabilities, however, we need to address three key challenges.

1) *Feature prototyping: Can we develop a trace-driven RTL CPU with minimal modifications to the existing execution-driven microarchitecture while preserving performance accuracy?* Our key insight is that hardware module interfaces can be categorized as functional interfaces, which determine what each instruction computes and where execution proceeds next, or performance-sensitive interfaces, which determine how efficiently the instruction stream is realized. Based on this distinction, TRACERTL selectively takes over key interfaces to decouple the functional model and maintain its performance behaviors with externally supplied traces. This preserves cycle-accurate performance accuracy while eliminating the complexity of managing full functional correctness.

2) *Broader workloads: Can we bridge the semantic gap*

across diverse trace formats and ISAs? Industrial workloads are valuable for microarchitecture exploration, but the scarcity of RISC-V workloads necessitates cross-ISA transformation to generate benchmark traces. This transformation is performed only once during trace preparation. However, differences in trace formats and ISAs hinder the direct execution of publicly available traces on RTL CPU models. Since trace-driven simulation relaxes the need for full functional correctness, TRACERTL introduces TraceBridge, a trace transformation technique that leverages instruction and register mapping to enable replaying traces from different formats and ISAs.

3) *Fast simulation: Can we accelerate the warm-up phase to improve evaluation efficiency?* Unlike execution-driven inputs, traces embed richer information, such as memory addresses and instruction types, thereby opening up new opportunities for optimization. We observe that warm-up inputs in traces often cause highly similar microarchitectural effects that lead to redundant executions, such as repeated accesses to the same cache lines. To exploit this redundancy, TRACERTL integrates TraceDedup, a fast warm-up strategy that identifies and clusters similar trace inputs to eliminate unnecessary executions and reduce warm-up time while preserving simulation fidelity.

To demonstrate the feasibility of TRACERTL, we develop a trace-driven RTL model derived from XiangShan [51], [70]. It achieves performance accuracies of 99.87% and 99.86% on SPECint2017 and SPECfp2017 respectively, reducing performance discrepancies by $10.31\times$ and $29.21\times$ compared to a calibrated XS-gem5 model. By leveraging TraceBridge, we evaluate x86-based Google workload traces [2] on XiangShan, and reveal distinct memory-bound behavior, compared to SPECint2017. Furthermore, TraceDedup delivers $1.5\times$ – $11.8\times$ simulation speedup of warm-up phases.

TRACERTL expands the possibilities for microarchitecture research by supporting both RTL-based exploration and seamless integration with simulator-based workflows. By preserving a simulator-like, trace-driven environment for workloads and simulations, it effectively bridges early-stage exploration on simulators and last-mile RTL evaluation. It is open-source to facilitate community collaboration and future development¹.

To summarize, this paper makes the following contributions.

- We propose TRACERTL, bringing trace-driven simulation to RTL CPUs for agile microarchitecture exploration.
- We propose TraceBridge, enhancing trace compatibility to expand the sources of benchmark workloads.
- We propose TraceDedup, enabling fast warm-up by eliminating redundant memory accesses and instructions.
- We demonstrate TRACERTL, using x86 workload traces collected from Google warehouse-scale computers for performance evaluation of XiangShan, a RISC-V CPU.

II. BACKGROUND

A. Out-of-Order Microarchitecture

Modern CPUs improve performance primarily by exploiting parallelism and speculation. The frontend speculatively

¹<https://github.com/OpenXiangShan/XiangShan/tree/dev-tracertl>

fetches instructions under branch prediction, while the backend decodes, schedules, and issues them to execution units for computation and to memory units for data access.

The efficiency of this pipeline depends on several critical microarchitectural components. Branch prediction and instruction fetch determine the instruction supply rate. Execution pipelines and scheduling queues affect throughput. The memory hierarchy bridges the large speed gap between the CPU and DRAM by caching frequently used data. The memory management unit (MMU) accelerates address translation by caching recently used mappings near the CPU.

B. Exploration on RTL

While RTL models offer higher accuracy for design space exploration, directly evaluating performance on RTL presents several challenges, including the inflexibility of traditional hardware description languages, slow simulation speeds, and lack of open-source RTL processors. There has been a growing trend toward addressing these issues.

Flexibility. Many emerging high-level hardware description languages [7], [13], [63] offer enhanced expressiveness and parameterization that accelerate the development of microarchitectures. New hardware design methodologies [74] are also proposed to further improve design modularity.

Simulation Speed. Novel RTL-simulation techniques have been proposed to accelerate software- [66], [68], [76] or hardware-based [25], [27], [37] simulation of RTL designs. On the other hand, sampling-based methods [18], [56], [62] estimate full-program performance by aggregating results from several representative program segments.

Open-Source RTL Processors. With the rapid growth of the RISC-V open-source community, a number of RTL processors have emerged, including in-order designs [6], [55], as well as out-of-order designs such as BOOM [19], [20], [75], XuanTie-910 [21], and XiangShan [70]. These designs provide accessible and realistic platforms for microarchitecture research, enabling agile exploration directly on RTL.

C. Simulation Methodologies

In computer architecture research, performance evaluation of novel designs predominantly relies on two core methodologies: execution-driven and trace-driven simulations. These approaches fundamentally differ in how they provide program stimuli to performance models, leading to distinct trade-offs between fidelity, flexibility and simulation speed.

The execution-driven methodology emulates the behaviors of real CPUs within the performance model, such as fetching, decoding, scheduling and executing instructions. This approach is inherent to RTL models [6], [19], [20], [70], [75] and is also implemented in many software simulators [11], [12], [16], [44]. By coupling functional execution with performance modeling, this approach captures microarchitecture-dependent dynamic behaviors, such as speculative execution and wrong-path effects, thereby offering high fidelity. However, this accuracy comes at the cost of significant complexity, being more error-prone, and reduced speed.

In contrast, the trace-driven methodology decouples the functional model from performance model by replaying the pre-generated trace of instructions including architectural information such as instruction semantics, instruction addresses, memory accesses, and branch outcomes [3], [17], [22], [33], [57]. These traces are often generated using instrumentation tools like Pin [42], DynamoRIO [14], and Valgrind [46], or from public pre-generated traces [1]–[3]. This decoupling affords higher flexibility, enabling researchers to focus on microarchitectural optimization. However, this flexibility often comes at the cost of reduced fidelity, as the trace lacks dynamic microarchitecture-dependent information.

III. CHALLENGE

Agile performance evaluation requires fast feature prototyping, support for extensive workloads, and fast simulation. To meet these goals at the RTL level, trace-driven simulation offers a promising approach by decoupling performance and functional models and supporting trace-based workloads. However, integrating trace-driven simulation into existing execution-driven CPU RTL models introduces non-trivial challenges. Publicly available traces often vary in trace format, lack information like instruction encodings, and are sometimes generated from different instruction sets. Moreover, the warm-up phase, necessary to mitigate the cold-start effect, prolongs the simulation time.

A. RTL Integration and Fidelity

Transforming a complex execution-driven CPU RTL model into a trace-driven implementation presents unique challenges compared with building an RTL model from scratch or driving individual RTL modules independently. In addition to supplying stimuli to existing RTL modules, a trace-driven model must precisely control the instruction flow based on external traces while maintaining high performance fidelity.

Nevertheless, trace-driven simulation inherently faces performance-accuracy challenges due to the absence of speculative execution information in traces. For example, traces omit instructions from mispredicted paths, which in a real processor would still affect microarchitectural state such as caches. The lack of these side-effects can lead to discrepancies between trace-driven and execution-driven performance measurements.

B. Trace Compatibility

Trace-driven approaches can bypass the limitations of simulated peripheral environments, thereby enhancing the coverage of supported workloads. However, due to confidentiality constraints, instruction source code is often unavailable for publicly accessible trace files [1]–[3]. Another scenario involves target applications requiring evaluation that have not been adapted to the target instruction set, rendering direct assessment infeasible.

Instruction sets share commonalities but also exhibit significant differences, which hinder direct trace porting. For example, differences in general-purpose register conventions, instruction encodings and sizes, PC alignment rules, and the

range of direct branch instructions all impose constraints on cross-instruction-set trace evaluation. These challenges are particularly pronounced for RTL models, which typically lack sufficient abstraction capabilities.

C. Fast Warm-up on RTL Model

To improve performance simulation efficiency, it is essential to accelerate the warm-up phase. This phase initializes caches and branch predictor. However, many memory accesses and instructions offer redundant warming effects. These redundant operations result in ineffective simulation efforts, ultimately prolonging the overall simulation time.

Conventional simulators implement fast warm-up by dynamically refining the simulated model. For instance, ChampSim [30] accelerates warm-up by dynamically eliminating register dependencies of instructions and latencies of scheduling, execution and memory access. gem5 [11] provides a switch mechanism between a KVM (Kernel-based Virtual Machine) virtual CPU model and the target CPU. However, unlike software simulators, RTL models lack built-in mechanisms for dynamic modification or flexible runtime model switching.

IV. TRACERTL DESIGN

To enable agile performance evaluation of RTL designs, we first propose a trace-driven simulation methodology on RTL while preserving high accuracy (§ IV-A). Building on this, we introduce TraceBridge, a trace transformation method that enhances compatibility by enabling the replay of traces from different formats and instruction sets (§ IV-B). Moreover, we develop a fast warm-up strategy, TraceDedup, to eliminate redundant instructions and memory accesses (§ IV-C).

A. Trace-Driven Microarchitecture Design

We decompose the CPU into core components and describe how each component is stimulated by the trace. An interface, the set of I/O signals between modules, can be driven to control the module's behavior. By driving the key interfaces with the information in traces, TRACERTL replaces the functional model with external traces while maintaining the original performance behaviors. This section describes the design of trace-driven integration, to meet its objectives: (1) driving RTL modules with external instruction traces, (2) enforcing the CPU model to conform to the traces' instruction flow, and (3) identifying and mitigating performance discrepancies inherent in trace-driven simulation.

1) *Trace-Driven RTL Modules*: Our key insight is that hardware module interfaces can be classified into functional interfaces, which determine what each instruction computes and where execution proceeds next (e.g., arithmetic, branching, or exception handling), and performance-sensitive interfaces, which determine how efficiently the instruction stream is realized (e.g., branch prediction, cache access, and memory prefetching). Based on this distinction, we analyze key module behaviors and drive performance-sensitive modules using external instruction traces, preserving original performance characteristics without requiring full functional execution.

Branch predictor. The branch predictor's performance-critical interfaces primarily include two types: training and prediction. The predictor is trained on the committed branch outcomes. Therefore, mis-speculated path instructions, which are flushed from the pipeline, leave no side-effects. By substituting the branch outcomes with trace information, which includes branch direction and target, we are able to stimulate the training process. For prediction, the predictor takes the current program counter (PC) and branch history to generate the next instruction fetch request. While prediction is at speculative stage, the PC and history for correct-path instructions are consistent between execution-driven and trace-driven simulations.

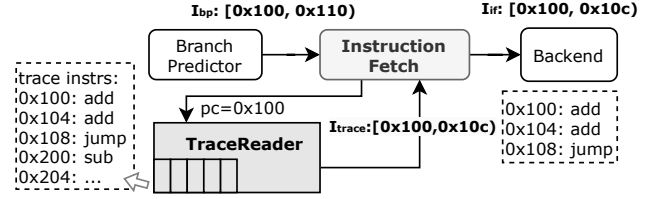


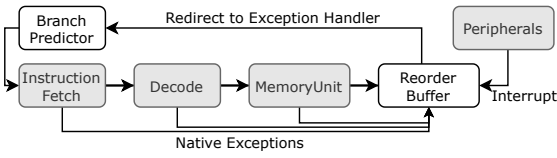
Fig. 2: Trace-driven instruction fetch.

Instruction fetch. Instruction fetch unit obtains fetch requests from the branch predictor and retrieves instructions from the traces. We propose an *interval match mechanism* to simulate the fetch bandwidth, as shown in Fig. 2. A fetch request typically specifies a contiguous instruction interval I_{bp} defined by starting and ending addresses. The fetch unit forwards the request to TraceReader that extracts a continuous sequence of trace instructions I_{trace} . Instructions in I_{if} , which are common to both I_{bp} and I_{trace} , are then sent to subsequent pipeline stages for execution. When the starting address does not match the beginning of the trace, I_{if} is empty, preventing any instructions from being fetched. Consequently, the impact of instructions on the mis-predicted path cannot be modeled. § IV-A3 presents a refined design to address this limitation.

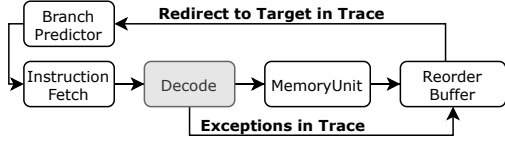
Out-of-order backend. The backend relies on instruction encodings to stimulate decoding, register renaming, dynamic scheduling and execution. These encodings are directly supplied from the trace. Alternatively, a more aggressive approach is to provide the results of the decoding directly to drive renaming and scheduling, although this is beyond the scope of this work. Particular units like the FDivSqrt operation may need optional data for accurate execution latency.

Cache hierarchy. Cache behavior is mainly influenced by access addresses. Instruction addresses are derived from fetch requests generated by the branch predictor. Data addresses, on the other hand, are dynamically calculated from the operands, which is invalid in trace-driven simulation. Therefore, memory access addresses should be included in traces to model memory behavior. Special modules like the indirect memory access prefetcher need extra data information.

Memory management unit. The virtual-to-physical address translation and page-table walk require in-memory page table entries (PTEs) that are typically absent in traces [23].



(a) Native exceptions and interrupt in the CPU pipeline.



(b) Intercept the native exceptions and trigger trace exceptions as illegal instruction.

Fig. 3: Exception and interrupt management in TRACERTL.

We employ a dynamic page table generation approach: For each instruction in the traces, we traverse the page tables using its virtual address. If a required PTE is invalid, a new page is allocated, and the corresponding PTE is initialized. This process continues recursively until reaching the leaf page, which is initialized with the physical address in traces.

2) *Trace-Controlled Instruction Flow*: TRACERTL controls the instruction flow by managing branch instructions, interrupts, and exceptions, while ensuring processor compliance by instruction stream correctness checks.

Branch instruction. We replace branch execution unit’s outcomes with target and conditional result recorded in the trace to control the programs’ instruction flow.

Exception and interrupt. Traps, including exceptions like page fault, and interrupts like timer interrupt, may be triggered by programs, devices, and operating system. Traps affect control flow and pipeline redirection, as illustrated in Fig. 3a. These are intercepted and re-injected according to the trace. Specifically, trace-recorded exceptions are triggered as illegal instructions, redirecting to the target in trace, as illustrated in Fig. 3b. This design ensures that exceptions are preserved without relying on full functional execution.

Instruction stream check. A fundamental requirement of trace-driven simulation is that the performance model must be guided by the trace, a key aspect of which is ensuring its execution adheres to the provided instruction stream. We capture the processor’s actual instruction stream through committed instructions and compare it against the trace. The differences in the streams indicate implementation flaws in the RTL model itself or trace-driven framework.

3) *Trace-Driven Performance Accuracy*: Trace-driven methodology introduces performance inaccuracies primarily due to information loss on two main fronts. First, accuracy is contingent on trace quality which is determined by the instruction instrumentation tools and publishers that omit precise instruction encodings [2], [3], privileged paths [36], etc. Second, trace-driven methodology has inherent flaws due to the lack of microarchitectural speculative information. For example, the

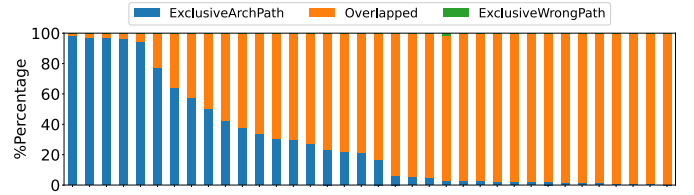


Fig. 4: Memory interval percentage weighted by load access times of SPEC CPU2017, sorted by “ExclusiveArchPath”.

trace lacks wrong path instructions, which are dynamically introduced during execution on a specific microarchitecture and thus cannot be pre-generated. Modern branch predictors often train only on the committed path, thus preventing wrong-path effects. Pipeline flushes on misprediction also clear transient states. Thus, wrong path instructions mainly pollute cache states, causing both positive and negative effects, which we categorize as **prefetching** and **pollution**.

To analyze the impact on caches, we perform a statistical study of both correct and wrong paths in the out-of-order processor XiangShan, focusing on instructions sent to the load pipeline. After aligning these addresses to cache line size, we categorize the address space into three types: (1) exclusive-arch-path: only accessed by correct path instructions, (2) exclusive-wrong-path: only accessed by wrong path instructions and (3) overlapped: accessed by both paths. As shown in Fig. 4, each bar represents a sub-benchmark with an input of SPEC CPU2017. We find that most of the address space falls into types (1) and (3). Therefore, we can tentatively draw a rough conclusion that **prefetching is the predominant influence**.

Based on the observation, we focus on simulating the prefetching influence by *taking the instructions at correct path as wrong-path instructions*. The process involves the following steps: (1) When a branch misprediction occurs, we check whether the fetch request’s starting address exists in traces within a fixed instruction window; (2) If it exists, the instructions in the trace are sent to subsequent pipeline stages as wrong path instructions. The instruction fetch unit is blocked for simplification. These instructions are not discarded from the traces; (3) Once the branch instruction is resolved and the pipeline is redirected, the correct fetch request is issued.

4) *Overall*: In summary, TRACERTL provides a general and adaptable framework for trace-driven RTL performance evaluation. It is designed to evolve naturally with RTL designs, require minimal effort across microarchitectural iterations, remain applicable across diverse microarchitectures, and flexibly support various performance optimizations.

Extending TRACERTL to new architectures. We summarize the trace-driven transformation methodology in Table I. It is applicable to both in-order and out-of-order processors, making TRACERTL a reusable and microarchitecture-agnostic framework for RTL performance evaluation. In-order cores reduce TRACERTL integration complexity due to their simplified fetch and execution behavior.

TABLE I: Key module behaviors and their corresponding trace-driven stimuli in TRACERTL.

Module	Key Behavior	Trace-Driven Stimulus
Branch Predictor	Prediction uses PC and history; Training uses committed branch outcomes.	Use current PC and history for prediction; Use branch outcomes from trace for training.
Instruction Fetch	Fetch request defines instruction interval; Wrong Path instructions.	Apply interval match mechanism to simulate fetch bandwidth; Generate wrong paths on mismatch.
Instruction Execution	Decode, rename, schedule, and execution.	Use instruction encoding and optional operand from trace.
Instruction Flow	Branch instruction outcome; Exception and interrupt redirect the pipeline.	Intercept branch outcomes/exception generation; Support redirect; Flow check.
Cache Hierarchy	Access cache by addresses.	Memory address from trace; Instruction address from branch predictor; Optional data from trace.
MMU	Virtual-to-physical address translation; Access memory for page table entries.	Construct page table according to the address from trace.

Applicability for microarchitecture features. TRACERTL is particularly advantageous for evaluating functionally complex yet performance-critical features (§VII-B). Beyond functionality, it captures fine-grained timing effects that are difficult to model accurately at higher abstraction levels. For example, variations in microarchitectural timing may critically affect the overall performance (§VII-C). It can also evaluate microarchitectural optimizations in the same way as conventional trace-driven simulators (e.g., branch prediction, prefetching, replacement, memory dependence prediction). With additional trace information, TRACERTL can be extended to model advanced optimizations, such as value prediction (with execution results) and indirect memory prefetching (with memory values).

B. Trace Compatibility with TraceBridge

We introduce trace transformation methodology TraceBridge to bridge the incompatibilities in trace formats and instruction sets. To support trace-driven simulation, the trace must contain at least three categories of information: (1) primary instruction type, including branch types, computation, and memory operations; (2) execution guidance, including PC, branch target and conditional result, and memory address; (3) register dependencies to model instruction-level parallelism. Such information is typically included in the trace format of dynamic instrumentation tools [23] and publicly available traces [1]–[3], where fine-grained semantic information such as instruction opcodes are sometimes missing.

TraceBridge retains the key information from the trace, transforming its format to be compatible with the target model by refining the execution semantics. However, trace-driven RTL models pose additional low-level challenges due to their rich details: (1) *instruction correspondence and register semantics*; (2) *difference in instruction encoding size and program counter (PC) alignment constraints*; (3) *variations in branch offset ranges*.

The primary principle of TraceBridge is to maintain performance semantic consistency. This ensures that the performance characteristics of the original program are reflected in the target architecture. For confidentiality, public traces omit instruction encodings [3] or provide instruction categories [2]. To address this, we observe that an instruction can encompass multiple performance semantics, which fall into four types: {Load, Computation, Store, Branch}. To maintain performance

semantic consistency, we map each individual performance semantic to its corresponding instruction(s) in the target ISA.

A single x86 instruction, which may encompass multiple micro-operations, is translated into an equivalent sequence of RISC-V instructions. For instance, the x86 RET instruction is mapped to two RISC-V instructions (LOAD and JR), and x86 memory accesses exceeding the width of a single RISC-V instruction are decomposed into multiple instructions to preserve the access range. The necessary mapping results in instruction inflation, which is analyzed in § VII-A. In the case of missing opcodes, compute instructions are mapped to representative types such as [F]ADD, [F]MUL, and CONVERT due to limited information in the traces. For ISAs with flag mechanism, such as x86, spare registers can be employed to establish inter-instruction dependencies. Furthermore, special handling for architecturally significant registers, like the return address register, guarantees the correct correspondence between x86 call/return operations and their RISC-V counterparts.

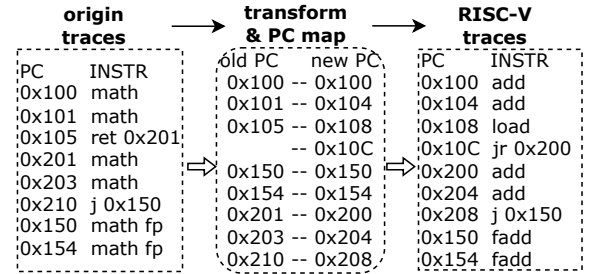


Fig. 5: Example of trace transformation.

To resolve differences in instruction size, PC alignment and instruction inflation, we reorganize PCs in the traces to conform to RISC-V requirements. As illustrated in Fig. 5, we collect all instruction PCs and sequentially reassign new addresses based on RISC-V encoding size. When a PC gap is detected (e.g., from 0x105 to 0x150), the current PC is updated accordingly. A mapping from original PCs to RISC-V PCs is then constructed, and branch target addresses are updated using this mapping.

While x86 ISA supports larger offset ranges for direct branch instructions than RISC-V, we observe that branch target computation mainly occurs in two modules: the pre-decoding unit at the fetch stage and the branch execution unit. By

overriding the computation result with the target recorded in traces, we effectively support larger branch offset ranges in the trace-driven RISC-V model.

Overall. TraceBridge provides a methodology to evaluate the microarchitectural behavior of mature, real-world software ecosystems (e.g. Google workloads) on an emerging hardware ecosystem (e.g. RISC-V). Admittedly, TraceBridge is unable to eliminate all performance discrepancies caused by inherent cross-ISA differences and missing execution information in traces, such as instruction semantics and ABIs. Furthermore, while the high-level methodology is consistent, the specific rules should adapt for source and target ISAs. For example, x86 and RISC-V differ in the number of general-purpose registers. Consequently, when translating to the x86, some registers may map to memory (i.e., register spilling). It is also constrained by information missing from the trace, forcing a simplified instruction re-mapping, which inevitably introduces performance errors. However, according to our evaluation on missing RISC-V opcodes (§VII-A), its accuracy is above 99% (0.95% error for SPECint2017) for early-stage performance exploration.

C. Fast Warm-up Strategy TraceDedup

We propose TraceDedup, a fast warm-up strategy that uses signature-based deduplication to simultaneously warm up the branch predictor and caches without requiring knowledge of their internal designs. TraceDedup optimizes three key aspects: (1) *eliminating instruction scheduling and execution*, (2) *minimizing memory accesses*, and (3) *reducing instructions*. It groups inputs with equivalent microarchitectural effects into signatures. For example, memory accesses to the same cache line share a single signature. By detecting and eliminating redundant warm-up operations through signature matching, TraceDedup significantly accelerates the warm-up phase.

For out-of-order scheduling and execution, TraceDedup eliminates instruction execution latency by marking arithmetic and memory access instructions as completed directly to bypass all the out-of-order scheduling, computation and memory access. For branch instructions, we leverage the branch instruction results recorded in trace to determine prediction correctness immediately after instructions leave the fetch unit and preemptively redirect the pipeline, thereby reducing branch misprediction penalties.

To accelerate cache warm-up, TraceDedup eliminates redundant memory accesses by leveraging cache-line-based signatures. Memory requests to the same cache line are assigned the same signature. During the warm-up phase, we compute signatures for all addresses and retain only the youngest address per signature. These selected addresses access the L1 data cache via its prefetch port, significantly reducing the number of memory requests. A detailed warm-up after the fast warm-up phase is needed to warm up data prefetch and replacement policy.

To accelerate branch predictor warm-up, TraceDedup eliminates redundant instructions. We utilize the training input (program counter, global history, target, taken) as signature for

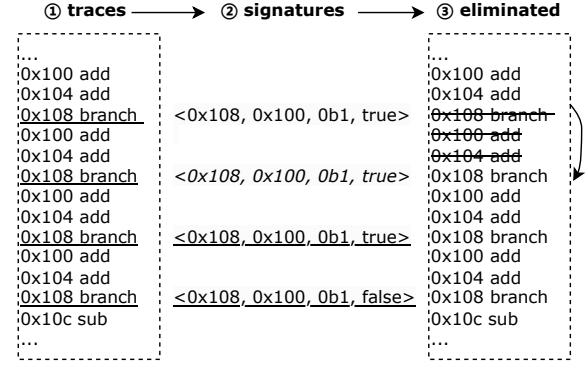


Fig. 6: Instruction elimination of TraceDedup.

branch instructions. Unlike caches, branch predictor imposes two unique requirements: (1) Each signature needs several duplications to train predictors; (2) Global history dependencies mandate preserving sufficient older branch instructions to maintain correct history.

Fig. 6 illustrates the process of redundant instruction identification and elimination: ① Branch instructions in the warm-up phase are traversed sequentially to extract their signatures. ② Unique signatures are marked in reverse order (the third and fourth branches). For each signature, a number of branch instructions, equal to the history length, are preserved (the second branch). ③ The remaining instructions are labeled as redundant, forming continuous redundant intervals. Each eliminated interval requires identical start and end instructions to enable seamless elimination.

V. PUT IT ALL TOGETHER

TRACERTL improves the performance evaluation workflow by optimizing stages such as workload preparation, prototyping, and performance simulation. As illustrated in Fig. 7, a typical iterative workflow based on TRACERTL is employed to perform agile performance evaluation. The workflow involves the following steps: ① **Trace Preparation:** Program traces for the benchmarks or target applications are prepared for subsequent performance evaluation. Each trace represents a program segment. Traces can be generated using a variety of tools, including dynamic instrumentation tools like Pin [42] and DynamoRIO [14], instruction-level simulators like QEMU [10], and publicly available traces such as Google workload traces [2] and Qualcomm workload traces [1]. TRACERTL can be combined with additional techniques such as SimPoint [62] to further shorten simulation time, while also avoiding the overhead and complexity of booting. ② **Prototyping:** New microarchitectural features can be prototyped on RTL model without full implementation, as shown in §VII-B. ③ **Trace-Driven Simulation:** The trace-formatted program segments are replayed in trace-driven simulation, sped up by TraceDedup, yielding performance results of the CPU model. ④ **Performance Analysis:** The performance results and program behaviors are analyzed to

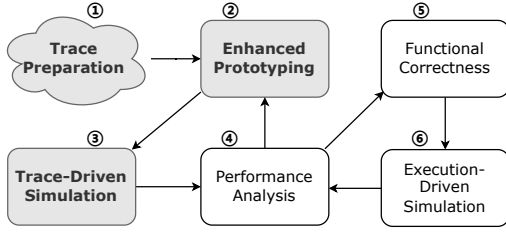


Fig. 7: Agile performance evaluation with TRACERTL.

identify performance bottlenecks. These insights inform subsequent iterations and guide prototype refinement. ⑤ **Functional Correctness**: When the design meets expected performance targets, the efforts invested in prototype development can be seamlessly carried over. TRACERTL supports compile-time mode switching between execution-driven and trace-driven simulation, enabling smooth transition to functional validation. ⑥ **Execution-Driven Simulation**: Further performance analysis and iteration are conducted through execution-driven.

TRACERTL facilitates an agile and accurate RTL-level microarchitecture design exploration process. Rather than replacing existing architectural simulators, TRACERTL serves as a complementary and reinforcing component that enhances RTL performance exploration and bridges the gap between high-level models and real RTL behavior. It targets a distinct sweet spot in the accuracy-productivity trade-off, preserving the ground-truth RTL model and accepting manageable maintenance overhead to achieve substantially higher accuracy, with comparable or potentially lower (Palladium/FPGA) simulation cost. By enabling direct performance evaluation on real RTL implementations, TRACERTL empowers architects to broaden application coverage and identify microarchitectural bottlenecks that high-level simulators may overlook.

VI. EVALUATION

We conduct evaluations to address three key questions:

- 1) Can our wrong-path simulation strategy mitigate trace-driven simulation’s performance inaccuracies?
- 2) Does TRACERTL achieve high performance accuracy?
- 3) Does TraceDedup reduce simulation time during warm-up phases?

To address these questions, we compare the performance of the original RTL model, TRACERTL, and the state-of-the-art simulator gem5 [11]. We then evaluate fast warm-up strategy TraceDedup’s speed-up effect and performance accuracy.

A. Experimental Setup

Target System. We evaluate TRACERTL by altering an open-source high-performance RISC-V processor, XiangShan [51], [70], into a trace-driven model. TRACERTL introduces low implementation overhead while preserving RTL fidelity. It reuses the original RTL and drives existing modules by intercepting inputs/outputs. Trace-driven integration achieves <1.5% Chisel LOC (line of code) modifications compared to

TABLE II: Target system configuration.

Component	Description
Branch Predictor	uBTB, BTB, TAGE-SC, ITTAGE, RAS
Fetch/Decode/Rename Width	8/6/6
RoB/LoadQueue/StoreQueue	160/72/64
Integer/Float Register File	224/192
ALU/FMA/FDivSqrt unit	4/4/2/
Load/Store unit	3/2
L1 ICache	64KB, 4-way, 256-set
L1 DCache	64KB, 8-way, 128-set
L2 Cache	1MB, 8-way, 512-set, 4-bank
L3 Cache	16MB, 16-way, 4096-set, 4-bank
L1 ITLB/DTLB	48-entry, fully-associative
L2 TLB	2048-entry, 8-way, 32-set
DRAM	DRAMsim3, 8GB, DDR4-3200

XiangShan. Particularly, modifications for interface connections and execution guidance to existing modules are <450 LOC.

XiangShan, implemented in Chisel [7], is a tape-out ready superscalar out-of-order processor. Its latest third-generation, Kunminghu, achieves a clock frequency of 3GHz and SPECint2006 score of exceeding 15/GHz, demonstrating its capability as a platform for exploring high performance microarchitecture designs. We use the default configuration of XiangShan, as shown in Table II. We take the original XiangShan’s performance as the ground truth.

gem5 is widely used for CPU microarchitecture exploration and is often referenced as the ground truth in some simulator works [24], [33], [53] for its rich details. We use the XS-gem5 [52] as the baseline, which has been carefully calibrated to XiangShan through over 1,200 git commits and more than 60,000 lines of source code additions since July 2022, including XiangShan-specific adjustments.

Simulation Speed. XS-gem5 achieves a simulation speed of around 35kHz. As TRACERTL is directly derived from the original RTL model, it inherently shares the comparable simulation speed and benefits from hardware-accelerated emulation tools. The simulation speeds are both around 6.5kHz using Verilator [66] and around 1.4MHz on Cadence Palladium, which is 40× faster than XS-gem5.

Workloads. We use SPEC CPU2006 [35] and SPEC CPU2017 [15] benchmark suites. We compare the benchmark scores between XiangShan, TRACERTL and XS-gem5. The complete execution of SPEC CPU benchmarks takes a very long time in software simulation. A set of representative program segments are generated by sampling the SPEC CPU benchmarks using SimPoint [62]. Each segment consists of 20M instructions for warm-up and 20M instructions for performance sampling. To limit simulation time, more than 30% weight of the program segments are included for each application. NEMU [50], an instruction-level simulator, is employed to execute these segments and generate trace files to feed into TRACERTL. Both XiangShan and XS-gem5 are functionally verified against NEMU, guaranteeing they share the same execution flow.

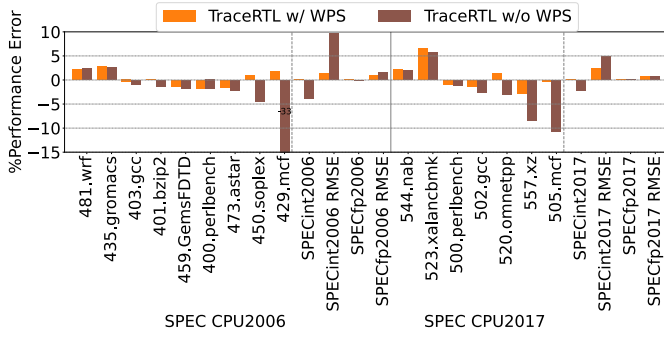


Fig. 8: Performance errors w/ and w/o wrong path simulation.

B. Wrong Path Simulation

For the first time, we evaluate the performance impact of the trace-driven simulation on an accurate high-performance RTL processor and the effectiveness of measures to mitigate the inaccuracy. We adopt the mechanism detailed in § IV-A3 to model wrong path effects. For comparison, we also consider the basic approach where the instruction fetch halts upon encountering a mis-prediction, detailed in § IV-A1. Fig. 8 illustrates SPEC CPU2006’s and SPEC CPU2017’s performance difference with and without simulating wrong path instructions’ effect, containing the sub-benchmarks whose “w/o WPS” errors are more than 1%, benchmarks’ overall performance errors and RMSE (root mean squared error) metric. Although the overall performance impact of neglecting wrong paths is relatively small (-3.91% and -0.18% for SPECint2006 and SPECfp2006, -2.17% and 0.14% for SPECint2017 and SPECfp2017), certain benchmarks, such as 429.mcf and 450.soplex on SPEC CPU2006 and 505.mcf and 557.xz on SPEC CPU2017, exhibited substantial performance degradation. Our results demonstrate that simulating the impact of wrong path instructions effectively mitigates these programs’ performance discrepancies, reducing the overall performance error to 0.14% for SPECint2006 and 0.13% for SPECint2017. The RMSE of SPECint2006 and SPECint2017 falls from 9.56% and 4.87% to 1.38% and 2.38%.

C. Overall Performance Accuracy

We evaluate the performance accuracy of TRACERTL and XS-gem5 on SPEC CPU2006 and SPEC CPU2017, with original XiangShan as the ground truth, as shown in Fig. 9.

Overall. TRACERTL achieves significantly high accuracy in overall performance. For RMSE metric, TRACERTL achieves 1.45% and 1.00% on SPECint2006 and SPECfp2006, compared to 9.85% and 19.44% for XS-gem5. Similarly, the RMSE of SPECint2017 and SPECfp2017 of TRACERTL are 2.38% and 0.67%, compared to 8.02% and 22.53% of XS-gem5.

Sub-benchmarks. TRACERTL exhibits high accuracy at both the overall and sub-benchmark levels. For XS-gem5, on SPEC CPU2006, 11 out of 29 sub-benchmarks have errors greater than 10%, and 14 out of 29 have errors greater than 3%. Similarly, on SPEC CPU2017, 7 out of 23 sub-benchmarks

have errors greater than 10%, and 13 out of 23 have errors greater than 3%. These discrepancies can be attributed to the diversity of program characteristics, which makes it challenging to perfectly calibrate. In contrast, by inheriting rich details, TRACERTL effortlessly achieves high accuracy. TRACERTL achieves performance accuracy that only 1 out of 29 on SPEC CPU2006 and 1 out of 23 on SPEC CPU2017 has an error greater than 3%.

D. Fast Warm-up Strategy TraceDedup

In this section, we evaluate the effectiveness of our fast warm-up strategy TraceDedup on RTL models. We primarily focus on two metrics: (1) speed-up ratio of the warm-up phase, defined as the ratio of clock cycles before and after applying fast warm-up, and (2) the performance accuracy of the sampling phase. For comparison, we use TRACERTL with conventional detailed warm-up as the baseline of performance and simulation time. For the warm-up phase, we employ TraceDedup to speed up the front 19-million-instruction warm-up, followed by a 1-million-instruction detailed warm-up. For comparison, we evaluate “Warmup-Cache” (cache pre-warming by addresses from front 19M-instructions plus 1M-instruction detailed warm-up) and “Warmup-1M-Insts” (only 1M-instruction detailed warm-up).

As shown in Fig. 10, 10 out of the 23 sub-benchmarks achieve speed-up exceeding $4\times$ for the warm-up phase, with 538.imagick exhibiting an $11.8\times$ speed-up. The SPECint2017 has an average speed-up of $3.0\times$ for the warm-up phase, while SPECfp2017 has an average speed-up of $5.4\times$. For changes in performance accuracy, the error increases to -1.50% for SPECint2017, and to -0.52% for SPECfp2017. Compared with “Warmup-1M-Insts” strategy, whose performance errors are -3.74% on SPECint2017 and -1.27% on SPECfp2017, TraceDedup reduces the errors by $2.49\times$ and $2.44\times$. The “Warmup-Cache” strategy accelerates the simulation time by $18\times$ to $43\times$ for the warm-up phase, but with only a marginal improvement in accuracy compared to “Warmup-1M-Insts” strategy. All three strategies exhibit slight memory performance degradation, which indicates insufficient warm-up. For branch predictor, however, “Warmup-Cache” and “Warmup-1M-Insts” increase bad speculation as they omit the predictor warm-up. In contrast, TraceDedup slightly improves branch prediction accuracy over the baseline, by reducing bad speculation.

VII. CASE STUDIES

In this section, we present case studies to demonstrate how TRACERTL facilitates agile performance evaluation:

- 1) **Trace Compatibility:** Using TraceBridge, we evaluate x86-based Google workload traces on the RISC-V XiangShan CPU (§VII-A).
- 2) **Prototyping:** We use TRACERTL to quickly evaluate the performance impact of adopting a two-stage address translation MMU (§VII-B).

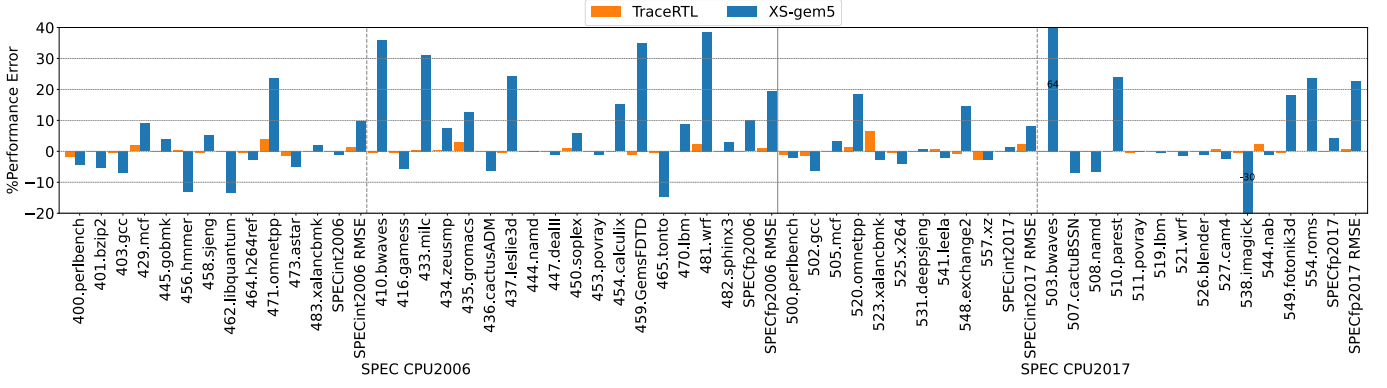


Fig. 9: Performance error of TRACERTL and XS-gem5.

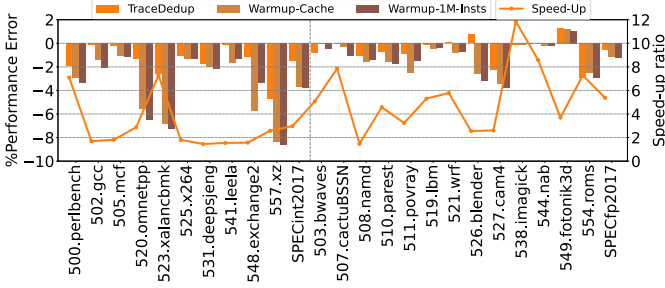


Fig. 10: Speed-up ratio and performance error of fast warm-up strategy on SPEC CPU2017.

- 3) **Performance Sensitivity Accuracy:** We compare the accuracy of performance impact between TRACERTL and XS-gem5 at frontend, backend and memory (§VII-C).

A. Trace Compatibility: Google Workload Traces

We evaluate datacenter workloads, the x86-based Google workload traces [2] from warehouse-scale computer workloads, on the RISC-V high performance processor XiangShan to show the feasibility of TraceBridge described in § IV-B. Google workload traces consist of multiple trace groups, each containing many trace files. For each group, we select the longest trace and apply the SimPoint [62] for sampling. Applying SimPoint directly to the transformed traces can avoid errors caused by instruction inflation.

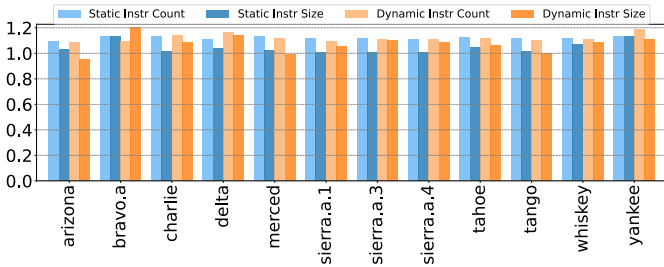


Fig. 11: Instruction inflation rate of TraceBridge on Google workload traces.

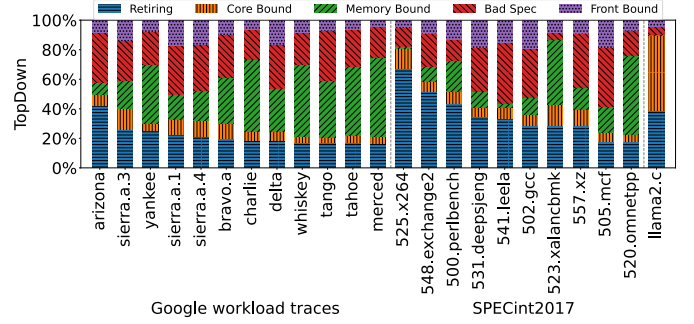


Fig. 12: Top-down breakdown comparison between Google workload traces and SPECint2017.

While TraceBridge maintains semantic consistency, it introduces the overhead of instruction inflation. We analyze this inflation across both static and dynamic dimensions, considering instruction count and size, as shown in Fig. 11. The inflation ratios for static and dynamic instruction counts remain stable within a narrow range, from 1.09 for arizona to 1.19 for yankee. The dynamic instruction size, an indicator of instruction cache pressure, exhibits an inflation ratio ranging from 0.95 for arizona to 1.20 for bravo.a, with 9 out of 12 applications staying within a 10% inflation margin.

We provide a TopDown breakdown analysis of performance bottlenecks in Google workload traces and SPECint2017, sorted by IPC, as shown in Fig. 12. While only 3 out of 10 SPECint2017 sub-benchmarks exhibit memory-bound over 20%, 8 out of 12 Google workload traces demonstrate this characteristic, with 6 reaching approximately 40%. These results highlight memory access as the primary performance bottleneck, underscoring the importance of memory optimization for warehouse-scale computing systems.

TRACERTL also streamlines porting workloads by leveraging the well-developed QEMU. It takes less than 30 minutes to compile llama2.c [39] and generate program traces by QEMU. As shown in Fig. 12, these traces are simulation on TRACERTL, and, unlike Google workload traces and SPECint2017, exhibits distinct core-bound behaviors.

To support traces without detailed opcodes, TraceBridge

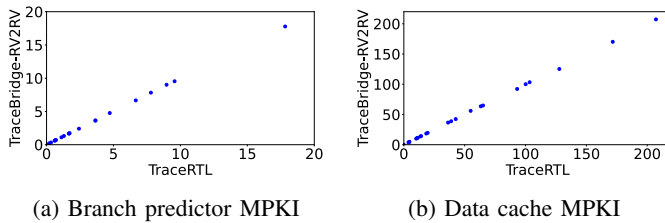


Fig. 13: BPU and D-Cache MPKI comparison between TraceRTL and TraceBridge-RV2RV on SPEC CPU 2017 benchmarks. Each point represents one sub-benchmark.

refines them with specific instruction opcodes. To evaluate the performance discrepancies caused by such refinement, we applied it in a (no-opcode) RISC-V to (refined) RISC-V translation scenario. As shown in Figure 13, despite missing and re-mapped opcodes, MPKI of the branch predictor and data cache confirms that TraceBridge maintains the fidelity of the branching and memory-access behaviors. However, this instruction type refinement leads to performance discrepancies (0.95% for SPECint2017 and 5.3% for SPECfp2017).

B. Prototyping: Memory Management Unit

TRACERTL enables efficient prototyping and performance evaluation of complex microarchitectural modules. As a case study, we examine two-stage address translation, a key mechanism for supporting virtual machines through memory virtualization defined in the RISC-V Hypervisor extension [54].

Evaluating this module is non-trivial due to its reliance on privileged operations, complex control and status registers (CSRs), and software-managed page tables. Additionally, its performance impact is significant: address translation may trigger multiple memory accesses to page table. For instance, the RISC-V Sv39 scheme requires 3 memory accesses, while the virtualized, two-stage Sv39-Sv39x4 scheme requires up to 15 memory accesses that increase the translation latency.

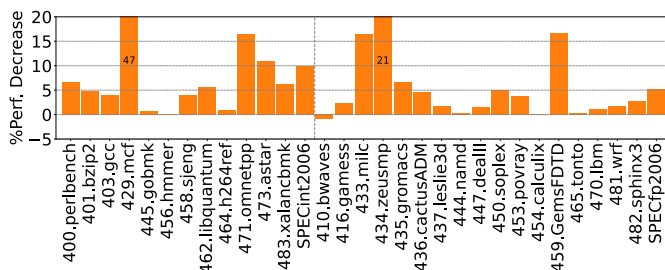


Fig. 14: Performance decrease when adopting two-stage address translation on SPEC CPU2006.

TRACERTL allows performance evaluation of such designs before full functional implementation is complete. By (1) directly providing the page table following the two-stage translation scheme and (2) adding a standalone host page table walker which performs guest-physical-address to host-physical-address translation, we enable the MMU to perform the two-stage Sv39-Sv39x4 scheme, thereby obtaining the

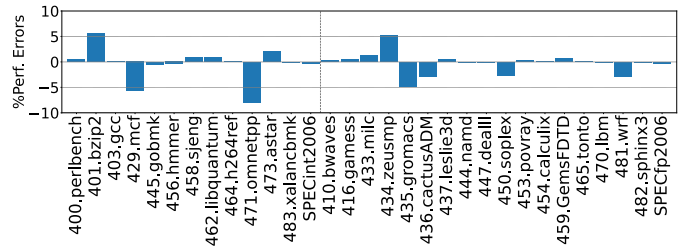


Fig. 15: Performance errors of TRACERTL on SPEC CPU2006 under KVM Virtualization.

performance results of two-stage address translation. Fig. 14 illustrates the performance changes of the TRACERTL under normal address translation and two-stage address translation modes on SPEC CPU2006. The two-stage address translation results in a performance degradation of 9.99% for SPECint2006 and 5.27% for SPECfp2017. Among the 29 sub-benchmarks, 10 have a degradation exceeding 5%. In summary, TRACERTL simplifies the requirements for functional correctness and software modifications, providing a robust development platform for exploration around MMU.

To evaluate the accuracy, we compare TRACERTL-based Hypervisor against fully-functional XiangShan Hypervisor on SPEC CPU2006 under KVM virtualization. As shown in Fig. 15, TRACERTL achieves high accuracy, with performance errors below 1% for 19 of 23 sub-benchmarks. The overall error is 0.32% for SPECint2006 and 0.23% for SPECfp2006.

C. Performance Sensitivity Accuracy

In addition to the performance accuracy of the processor model, the performance sensitivity to microarchitectural modifications is also important. To evaluate the performance sensitivity accuracy to microarchitectural modifications, we adjust key configurations in the frontend, backend, and memory subsystem. For the frontend, we compare the performance impact of different branch target buffer (BTB) sizes—specifically, from 1024 to 2048 (default) entries. For the backend, we vary the number of floating-point units FMA from 2 to 4 (default). For the memory subsystem, we evaluate performance with the Best-Offset prefetcher in the L2 cache both disabled and enabled (default).

As shown in Fig. 16, we compare the performance variations of XiangShan, TRACERTL and XS-gem5 on SPEC CPU2017 benchmarks under microarchitectural modifications mentioned above. The performance trends observed on TRACERTL closely match those of XiangShan better than those of XS-gem5. For instance, when enlarging BTB size, sub-benchmarks such as 500.perlbench, 502.gcc, and 511.povray exhibit similar trends between TRACERTL and XiangShan. When increasing the number of the FMA, sub-benchmarks like 507.cactuBSSN, 508.namd, and 519.lbm show consistent behavior. For adopting the best-offset prefetcher, sub-benchmarks including 500.perlbench and 507.cactuBSSN also demonstrate analogous performance improvements.

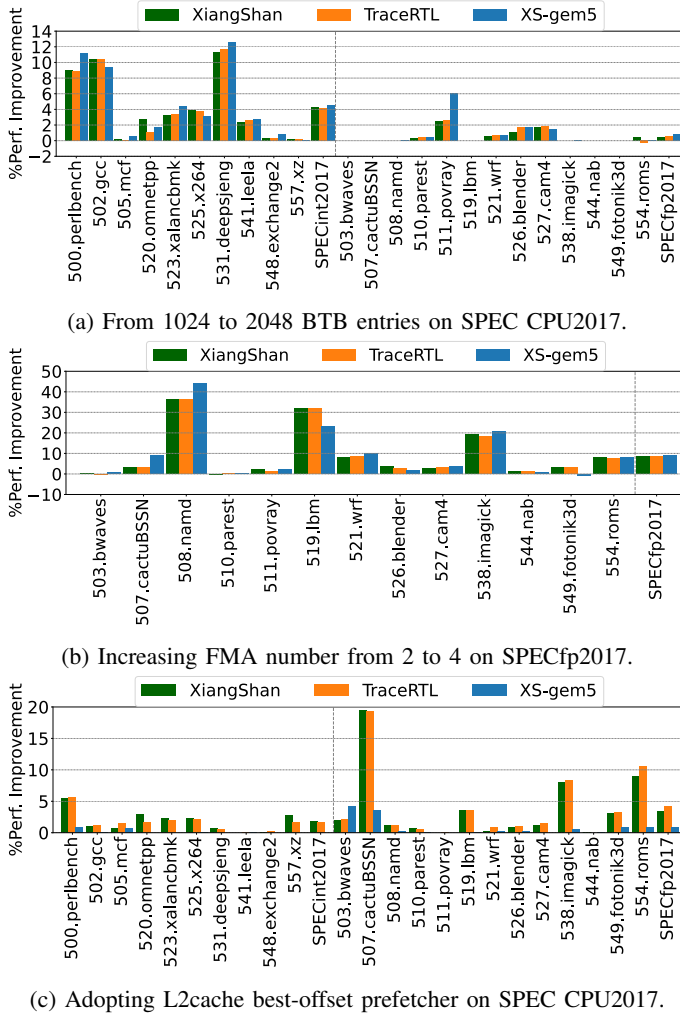


Fig. 16: Performance improvements of microarchitectural modifications on XiangShan, TRACERTL and XS-gem5.

We analyze the notable performance errors of XS-gem5 and find that its prefetching subsystem is considerably more complex and finely tuned, yet lacks clear calibration against the RTL design. This mismatch diminishes the observable performance gains from new prefetchers such as best-offset. The observation highlights the fundamental calibration challenge and motivates the design of TRACERTL: while a model may overfit to the baseline configuration to reproduce similar overall performance, its performance trends for specific microarchitectural features may diverge significantly.

VIII. RELATED WORK

Trace-Driven Model Transformation. Prior work has explored employing trace-based methods to directly control RTL modules' behavior for functional verification, coverage analysis, and performance validation [41], [65]. Some works collect the traces generated by CPU RTL models for coverage analysis [43]. In contrast, TRACERTL, centered on the whole CPU RTL model, addresses the challenges of design space exploration at RTL level. Trace-driven methodology can be

used to improve existing software simulators, such as the trace-driven gem5 mentioned at [5]. In contrast, TRACERTL enhances the RTL simulation to avoid extra model layers. Accel-Sim [40] adds a new frontend for GPGPU-Sim [9] to support trace-driven simulation. Unlike Accel-Sim's high-level GPU modeling, TRACERTL targets low-level RTL CPU models and addresses challenges of model calibration.

Trace-Driven Performance Inaccuracy. Previous works have investigated performance inaccuracy in trace-driven simulation, primarily focusing on the wrong-path simulation in single-core [28], [31], [45], multi-core [60], [61] and synchronization in multi-core simulation [32], [59]. Our methodology mainly focuses on prefetching influence of wrong paths by taking the instructions at the correct path as wrong-path instructions, to suit RTL model and achieve high accuracy.

Error-Prone RTL Model. New RTL languages such as Bluespec SystemVerilog [13], Chisel [7], and SpinalHDL [63] provides high expressiveness and abstraction to reduce design errors. Assassyn [72] introduces a high-level abstraction for asynchronous event handling of pipelined architectures and can generate a calibrated C++ simulator. TRACERTL presents an orthogonal approach to utilizing a trace-driven methodology to decouple the functional and performance models of existing CPU models and to expanding the scope of workloads.

Fast Warm-up Strategies. Conventional simulators accelerate the warm-up phase by modifying the simulation model. For example, ChampSim [3] decreases the instruction execution and memory access latency, Sniper [17] directly accesses the branch predictor and caches to avoid core model simulation, and gem5 [58] switches to a KVM virtual CPU model to speed up. CoolSim [48] and Delorean [47] select key memory accesses to warm up data caches. TraceDedup eliminates redundancy not only in memory accesses to warm up caches but also in instructions to warm up branch predictor.

IX. CONCLUSION

We propose TRACERTL, a methodology to bring trace-driven simulation to the CPU RTL model to facilitate agile performance evaluation. We evaluate TRACERTL by integrating it into XiangShan, achieving high performance accuracy of 99.87% and 99.86% on SPECint2017 and SPECfp2017. We propose trace transformation strategy TraceBridge and evaluate x86-based Google workload traces on RISC-V XiangShan. We propose fast warm-up strategy TraceDedup to speed up warm-up phase by $1.5\times$ – $11.8\times$.

ACKNOWLEDGEMENT

The authors would like to thank the anonymous reviewers for their valuable feedback and comments. This work is supported in part by the National Natural Science Foundation of China (Grant No. 62090022, 62090023, 62172388) and the Strategic Priority Research Program of Chinese Academy of Sciences (Grant No. XDA0320000, XDA0320300).

REFERENCES

- [1] “Championship value prediction,” <https://microarch.org/cvp1/>, Accessed: 2025-02-20.
- [2] “Google workload traces version 2,” <https://console.cloud.google.com/storage/browser/external-traces-v2>, Accessed: 2025-02-20.
- [3] “Cbp2025 simulator framework,” <https://ericrotenberg.wordpress.ncsu.edu/cbp2025-simulator-framework/>, Accessed: 2025-04-08.
- [4] H. Ando, “Performance improvement by prioritizing the issue of the instructions in unconfident branch slices,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 82–94.
- [5] S. Apostolakis, C. Kennelly, X. D. Li, and P. Ranganathan, “Necro-reaper: Pruning away dead memory traffic in warehouse-scale computers,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 689–703. [Online]. Available: <https://doi.org/10.1145/3676641.3716007>
- [6] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, and J. Koenig, “The rocket chip generator,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17*, vol. 4, pp. 6–2, 2016.
- [7] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniek, and K. Asanović, “Chisel: constructing hardware in a scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference*, 2012, pp. 1216–1225.
- [8] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, “Boom-explorer: Risc-v boom microarchitecture design space exploration framework,” in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [9] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, “Analyzing cuda workloads using a detailed gpu simulator,” in *2009 IEEE International Symposium on Performance Analysis of Systems and Software*, 2009, pp. 163–174.
- [10] F. Bellard, “QEMU, a fast and portable dynamic translator,” in *USENIX annual technical conference, FREENIX Track*, vol. 41, no. 46. California, USA, 2005, pp. 10–5555.
- [11] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, “The gem5 simulator,” *SIGARCH Comput. Archit. News*, vol. 39, no. 2, p. 1–7, Aug. 2011. [Online]. Available: <https://doi.org/10.1145/2024716.2024718>
- [12] N. Binkert, R. Dreslinski, L. Hsu, K. Lim, A. Saidi, and S. Reinhardt, “The m5 simulator: Modeling networked systems,” *IEEE Micro*, vol. 26, no. 4, pp. 52–60, 2006.
- [13] T. Bourgeat, C. Pit-Claudel, A. Chlipala, and Arvind, “The essence of bluespec: a core language for rule-based hardware design,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 243–257. [Online]. Available: <https://doi.org/10.1145/3385412.3385965>
- [14] D. Bruening, E. Duesterwald, and S. Amarasinghe, “Design and implementation of a dynamic optimization framework for windows,” in *4th ACM workshop on feedback-directed and dynamic optimization (FDDO-4)*, 2001, p. 20.
- [15] J. Bucek, K.-D. Lange, and J. v. Kistowski, “Spec cpu2017: Next-generation compute benchmark,” in *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*, 2018, pp. 41–42.
- [16] D. Burger and T. M. Austin, “The simplescalar tool set, version 2.0,” *SIGARCH Comput. Archit. News*, vol. 25, no. 3, p. 13–25, Jun. 1997. [Online]. Available: <https://doi.org/10.1145/268806.268810>
- [17] T. E. Carlson, W. Heirman, and L. Eeckhout, “Sniper: exploring the level of abstraction for scalable and accurate parallel multi-core simulation,” in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11. New York, NY, USA: Association for Computing Machinery, 2011. [Online]. Available: <https://doi.org/10.1145/2063384.2063454>
- [18] T. E. Carlson, W. Heirman, K. Van Craeynest, and L. Eeckhout, “Barrierpoint: Sampled simulation of multi-threaded applications,” in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2014, pp. 2–12.
- [19] C. Celio, P.-F. Chiu, B. Nikolic, D. A. Patterson, and K. Asanovic, “Boomv2: an open-source out-of-order risc-v core,” in *First Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.
- [20] C. Celio, D. A. Patterson, and K. Asanovic, “The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167*, 2015.
- [21] C. Chen, X. Xiang, C. Liu, Y. Shang, R. Guo, D. Liu, Y. Lu, Z. Hao, J. Luo, Z. Chen, C. Li, Y. Pu, J. Meng, X. Yan, Y. Xie, and X. Qi, “Xuantie-910: A commercial multi-core 12-stage pipeline out-of-order 64-bit high performance risc-v processor with vector extension: Industrial product,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 52–64.
- [22] R.-V. community, “Olympia,” <https://github.com/riscv-software-src/riscv-perf-model>, 2024.
- [23] DynamoRIO, “Dynamorio trace format,” https://dynamorio.org/sec_drcachesim_format.html, 2024.
- [24] M. E. S. Elrabaa, A. Hroub, M. F. Mudawar, A. Al-Aghbari, M. Al-Asli, and A. Khayyat, “A very fast trace-driven simulation platform for chip-multiprocessors architectural explorations,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 11, pp. 3033–3045, 2017.
- [25] F. Elsabbagh, S. Sheikha, V. A. Ying, Q. M. Nguyen, J. S. Emer, and D. Sanchez, “Accelerating rtl simulation with hardware-software co-design,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 153–166.
- [26] M. Emami, T. Bourgeat, and J. R. Larus, “Parendi: Thousand-way parallel rtl simulation,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 783–797. [Online]. Available: <https://doi.org/10.1145/3676641.3716010>
- [27] M. Emami, S. Kashani, K. Kamahori, M. S. Pourghannad, R. Raj, and J. R. Larus, “Manticore: Hardware-accelerated rtl simulation with static bulk-synchronous parallelism,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2023, pp. 219–237.
- [28] S. Eyerma, S. Van den Steen, W. Heirman, and I. Hur, “Simulating wrong-path instructions in decoupled functional-first simulation,” in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 124–133.
- [29] M. Ghaniyoun, K. Barber, Y. Xiao, Y. Zhang, and I. R. Teodorescu, “Teesecc: Pre-silicon vulnerability discovery for trusted execution environments,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589070>
- [30] N. Guber, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim, “The championship simulator: Architectural simulation for education and competition,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.14324>
- [31] B. R. Godala, S. P. Ramesh, K. Tibrewala, C. Pepi, G. Chacon, S. Kanev, G. A. Pokam, D. A. Jiménez, P. V. Gratz, and D. I. August, “Correct wrong path,” *arXiv preprint arXiv:2408.05912*, 2024.
- [32] S. R. Goldschmidt and J. L. Hennessy, “The accuracy of trace-driven simulations of multiprocessors,” *ACM SIGMETRICS Performance Evaluation Review*, vol. 21, no. 1, pp. 146–157, 1993.
- [33] H. Golestani, R. Sen, V. Young, and G. Gupta, “Calipers: a criticality-aware framework for modeling processor performance,” in *Proceedings of the 36th ACM International Conference on Supercomputing*, ser. ICS ’22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3524059.3532390>
- [34] S. Gupta, Y. Li, Q. Kang, A. Bhattacharjee, B. Falsafi, Y. Oh, and M. Payer, “Imprecise store exceptions,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589087>
- [35] J. L. Henning, “Spec cpu2006 benchmark descriptions,” *ACM SIGARCH Computer Architecture News*, vol. 34, no. 4, pp. 1–17, 2006.
- [36] K. Kanellopoulos, K. Sgouras, F. N. Bostanci, A. K. Kakolyris, B. K. Konar, R. Bera, M. Sadrosadati, R. Kumar, N. Vijaykumar, and O. Mutlu, “Virtuoso: Enabling fast and accurate virtual memory research via an imitation-based operating system simulation methodology,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume*

- 2, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1400–1421. [Online]. Available: <https://doi.org/10.1145/3676641.3716027>
- [37] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, D. Lee, N. Pemberton, E. Amaro, C. Schmidt, A. Chopra, Q. Huang, K. Kovacs, B. Nikolic, R. Katz, J. Bachrach, and K. Asanović, “Firesim: Fpga-accelerated cycle-exact scale-out system simulation in the public cloud,” in *Proceedings of the 45th Annual International Symposium on Computer Architecture*, ser. ISCA '18. IEEE Press, 2018, p. 29–42. [Online]. Available: <https://doi.org/10.1109/ISCA.2018.00014>
- [38] S. Karandikar, A. Ou, A. Amid, H. Mao, R. Katz, B. Nikolić, and K. Asanović, “Fireperf: Fpga-accelerated full-system hardware/software performance profiling and co-design,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 715–731. [Online]. Available: <https://doi.org/10.1145/3373376.3378455>
- [39] A. Karpathy, “llama2.c: Inference Llama 2 in one file of pure C,” <https://github.com/karpathy/llama2.c>.
- [40] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, “Accel-sim: An extensible simulation framework for validated gpu modeling,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 473–486.
- [41] L. Liu and S. Vasudevan, “Efficient validation input generation in rtl by hybridized source code analysis,” in *2011 Design, Automation & Test in Europe*, 2011, pp. 1–6.
- [42] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, “Pin: building customized program analysis tools with dynamic instrumentation,” in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '05. New York, NY, USA: Association for Computing Machinery, 2005, p. 190–200. [Online]. Available: <https://doi.org/10.1145/1065010.1065034>
- [43] B. Mammo, J. Larimer, M. Morgan, D. Fan, E. Hennenhoefler, and V. Bertacco, “Architectural trace-based functional coverage for multi-processor verification,” in *2012 13th International Workshop on Micro-processor Test and Verification (MTV)*, 2012, pp. 1–5.
- [44] M. M. K. Martin, D. J. Sorin, B. M. Beckmann, M. R. Marty, M. Xu, A. R. Alameldeen, K. E. Moore, M. D. Hill, and D. A. Wood, “Multifacet’s general execution-driven multiprocessor simulator (gems) toolset,” *SIGARCH Comput. Archit. News*, vol. 33, no. 4, p. 92–99, Nov. 2005. [Online]. Available: <https://doi.org/10.1145/1105734.1105747>
- [45] O. Mutlu, H. Kim, D. N. Armstrong, and Y. N. Patt, “An analysis of the performance impact of wrong-path memory references on out-of-order and runahead execution processors,” *IEEE Transactions on Computers*, vol. 54, no. 12, pp. 1556–1571, 2005.
- [46] N. Nethercote and J. Seward, “Valgrind: a framework for heavyweight dynamic binary instrumentation,” in *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 89–100. [Online]. Available: <https://doi.org/10.1145/1250734.1250746>
- [47] N. Nikoleris, L. Eeckhout, E. Hagersten, and T. E. Carlson, “Directed statistical warming through time traveling,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 1037–1049. [Online]. Available: <https://doi.org/10.1145/3352460.3358264>
- [48] N. Nikoleris, A. Sandberg, E. Hagersten, and T. E. Carlson, “Coolsim: Statistical techniques to replace cache warming with efficient, virtualized profiling,” in *2016 International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation (SAMOS)*, 2016, pp. 106–115.
- [49] T. Nowatzki, J. Menon, C.-H. Ho, and K. Sankaralingam, “Architectural simulators considered harmful,” *IEEE Micro*, vol. 35, no. 6, pp. 4–12, 2015.
- [50] OpenXiangShan, “Nemu,” <https://github.com/OpenXiangShan/NEMU>, 2024.
- [51] OpenXiangShan, “Xiangshan,” <https://github.com/OpenXiangShan/XiangShan>, 2024.
- [52] OpenXiangShan, “Xs-gem5,” <https://github.com/OpenXiangShan/GEM5>, 2024.
- [53] S. Pandey, A. Yazdanbakhsh, and H. Liu, “Tao: Re-thinking dl-based microarchitecture simulation,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 8, no. 2, pp. 1–25, 2024.
- [54] RISC-V, “Risc-v instruction set manual,” <https://github.com/riscv/riscv-isa-manual>, 2025.
- [55] B. Sá, L. Valente, J. Martins, D. Rossi, L. Benini, and S. Pinto, “Cva6 risc-v virtualization: Architecture, microarchitecture, and design space exploration,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [56] A. Sabu, H. Patil, W. Heirman, and T. E. Carlson, “Looppoint: Checkpoint-driven sampled simulation for multi-threaded applications,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 604–618.
- [57] D. Sanchez and C. Kozyrakis, “Zsim: fast and accurate microarchitectural simulation of thousand-core systems,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 475–486. [Online]. Available: <https://doi.org/10.1145/2485922.2485963>
- [58] A. Sandberg, N. Nikoleris, T. E. Carlson, E. Hagersten, S. Kaxiras, and D. Black-Schaffer, “Full speed ahead: Detailed architectural simulation at near-native speed,” in *2015 IEEE International Symposium on Workload Characterization*, 2015, pp. 183–192.
- [59] K. Sangaiah, M. Lui, R. Jagtap, S. Diestelhorst, S. Nilakantan, A. More, B. Taskin, and M. Hempstead, “Synchrotrace: Synchronization-aware architecture-agnostic traces for lightweight multicore simulation of cmp and hpc workloads,” *ACM Trans. Archit. Code Optim.*, vol. 15, no. 1, Mar. 2018. [Online]. Available: <https://doi.org/10.1145/3158642>
- [60] R. Sendag, A. Yilmazer, J. Yi, and A. Uht, “Quantifying and reducing the effects of wrong-path memory references in cache-coherent multiprocessor systems,” in *Proceedings 20th IEEE International Parallel & Distributed Processing Symposium*, 2006, pp. 10 pp.–.
- [61] R. Sendag, A. Yilmazer, J. J. Yi, and A. K. Uht, “The impact of wrong-path memory references in cache-coherent multiprocessor systems,” *Journal of Parallel and Distributed Computing*, vol. 67, no. 12, pp. 1256–1269, 2007, best Paper Awards: 20th International Parallel and Distributed Processing Symposium (IPDPS 2006). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731507000457>
- [62] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, “Automatically characterizing large scale program behavior,” in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS X. New York, NY, USA: Association for Computing Machinery, 2002, p. 45–57. [Online]. Available: <https://doi.org/10.1145/605397.605403>
- [63] SpinalHDL, “Scala based hdl,” <https://github.com/SpinalHDL/SpinalHDL>, 2024.
- [64] W. Su, A. Dhanotia, C. Torres, J. Gandhi, N. Gholkar, S. Kanaujia, M. Naumov, K. Subramanian, V. Andrei, Y. Yuan, and C. Tang, “Dcperf: An open-source, battle-tested performance benchmark suite for datacenter workloads,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1717–1730. [Online]. Available: <https://doi.org/10.1145/3695053.3731411>
- [65] V. M. Vedula, J. A. Abraham, J. Bhadra, and R. Tupuri, “A hierarchical test generation approach using program slicing techniques on hardware description languages,” *Journal of Electronic Testing*, vol. 19, pp. 149–160, 2003.
- [66] Verilator, “Verilator user’s guide,” <https://www.veripool.org/guide/latest/>, 2024.
- [67] D. Wang, M. Yan, Y. Teng, D. Han, H. Dang, X. Ye, and D. Fan, “A transfer learning framework for high-accurate cross-workload design space exploration of cpu,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [68] H. Wang and S. Beamer, “Reput: Superlinear parallel rtl simulation with replication-aided partitioning,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 572–585. [Online]. Available: <https://doi.org/10.1145/3582016.3582034>
- [69] H. Wang, T. Nijssen, and S. Beamer, “Don’t repeat yourself! coarse-grained circuit deduplication to accelerate rtl simulation,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ser. ASPLOS '24. New York, NY, USA:

Association for Computing Machinery, 2025, p. 79–93. [Online]. Available: <https://doi.org/10.1145/3622781.3674184>

- [70] K. Wang, J. Chen, Y. Xu, Z. Yu, Z. Zhang, G. Chen, X. Hu, L. Zhang, X. Chen, W. He, D. Tang, N. Sun, and Y. Bao, “XiangShan: An Open-Source Project for High-Performance RISC-V Processors Meeting Industrial-Grade Standards,” in *2024 IEEE Hot Chips 36 Symposium (HCS)*. Los Alamitos, CA, USA: IEEE Computer Society, Aug. 2024, pp. 1–25. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/HCS61935.2024.10665293>
- [71] L. Wang, X. Zhang, S. Wang, Z. Jiang, T. Lu, M. Chen, S. Luo, and K. Huang, “Asynchronous memory access unit: Exploiting massive parallelism for far memory access,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 3, Sep. 2024. [Online]. Available: <https://doi.org/10.1145/3663479>
- [72] J. Weng, B. Han, D. Gao, R. Gao, W. Zhang, A. Zhong, C. Xu, J. Xin, Y. Luo, L. W. Wills, and M. Canini, “Assassyn: A unified abstraction for architectural simulation and implementation,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1464–1479. [Online]. Available: <https://doi.org/10.1145/3695053.3731004>
- [73] Y. Xu, Z. Yu, D. Tang, G. Chen, L. Chen, L. Gou, Y. Jin, Q. Li, X. Li, Z. Li, J. Lin, T. Liu, Z. Liu, J. Tan, H. Wang, H. Wang, K. Wang, C. Zhang, F. Zhang, L. Zhang, Z. Zhang, Y. Zhao, Y. Zhou, Y. Zhou, J. Zou, Y. Cai, D. Huan, Z. Li, J. Zhao, Z. Chen, W. He, Q. Quan, X. Liu, S. Wang, K. Shi, N. Sun, and Y. Bao, “Towards developing high performance risc-v processors using agile methodology,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1178–1199.
- [74] S. Zhang, A. Wright, T. Bourgeat, and Arvind, “Composable building blocks to open up processor design,” in *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-51. IEEE Press, 2018, p. 68–81. [Online]. Available: <https://doi.org/10.1109/MICRO.2018.00015>
- [75] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, “Sonicboom: The 3rd generation berkeley out-of-order machine,” in *Fourth Workshop on Computer Architecture Research with RISC-V*, vol. 5, 2020, pp. 1–7.
- [76] K. Zhou, Y. Liang, Y. Lin, R. Wang, and R. Huang, “Khronos: Fusing memory access for improved hardware rtl simulation,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 180–193. [Online]. Available: <https://doi.org/10.1145/3613424.3614301>